Charts

Neena Jain[1,1] emBOLDen LLC

Abstract

Outlines the foundational principles for creating compelling and accurate data visualizations. These principles are grounded in cartographic theory, information design, and empirical research in visual perception.

Plain Language Summary

You've learned:

A practical tutorial for creating publication-quality charts and static visualizations using pandas and matplotlib.

1. Section 1: Getting Started

1.a. What You'll Learn

- Import and set up pandas and matplotlib
- Create basic plots directly from pandas DataFrames
- Customize figure appearance and styling
- Create multi-panel figures and subplots
- Export publication-quality images

1.b. Why Pandas + Matplotlib?

Pandas excels at:

- Loading and manipulating tabular data
- Quick exploratory data analysis
- Built-in plotting methods

Matplotlib excels at:

- Fine-grained control over every visual element
- Publication-quality output
- Integration with scientific Python ecosystem

Together, they provide a powerful, flexible approach to data visualization.

Published Apr 16, 2026

```
# Import essential libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import matplotlib.patches as mpatches

# Set style and font sizes for better-looking plots
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11
plt.rcParams['lines.linewidth'] = 2

# Configure Seaborn for improved aesthetics
sns.set_palette("husl")

# For better display in Jupyter
%matplotlib inline

print("Libraries imported successfully!")
```

2. Section 2: Creating Basic Plots with Pandas

2.a. Sample Dataset

First, let's create a simple dataset to work with:

```
# Create a sample geospatial dataset
np.random.seed(42)
years = np.arange(2015, 2023)

data = {
    'Year': years,
    'Forest_Area_km2': 50000 + np.random.normal(0, 1000, len(years)) -
years * 100,
    'Deforestation_Rate_%': 2.5 + np.random.normal(0, 0.3, len(years)),
    'Protected_Area_km2': 15000 + years * 200 + np.random.normal(0, 500,
len(years)),
    'Population': 1000000 + years * 50000 + np.random.normal(0, 10000,
len(years))
}

df = pd.DataFrame(data)
print("Sample Dataset:")
print(df)
```

2.b. Line Plot - Tracking Changes Over Time

```
# Simple line plot
plt.figure(figsize=(10, 6))
df.plot(x='Year', y='Forest_Area_km2', kind='line', marker='o',
color='green', linewidth=2)

plt.title('Forest Area Over Time', fontsize=14, fontweight='bold')
plt.xlabel('Year', fontsize=12)
plt.ylabel('Area (km²)', fontsize=12)
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

2.c. Bar Plot - Comparing Values

Bar plots are excellent for comparing categories or showing discrete values:

```
# Bar plot - subset of data
subset_years = df[df['Year'].isin([2016, 2018, 2020, 2022])]

plt.figure(figsize=(10, 6))
subset_years.plot(x='Year', y='Deforestation_Rate_%', kind='bar',
color='orangered')

plt.title('Deforestation Rate by Year', fontsize=14, fontweight='bold')
plt.xlabel('Year', fontsize=12)
plt.ylabel('Deforestation Rate (%)', fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

2.d. Scatter Plot - Showing Relationships

```
# Scatter plot with color gradient
plt.figure(figsize=(10, 6))
scatter = plt.scatter(df['Protected_Area_km2'], df['Forest_Area_km2'],
                      c=df['Year'], cmap='viridis', s=100, alpha=0.7,
edgecolors='black')

plt.title('Forest Area vs Protected Area', fontsize=14,
fontweight='bold')
plt.xlabel('Protected Area (km²)', fontsize=12)
plt.ylabel('Forest Area (km²)', fontsize=12)

# Add colorbar
colorbar = plt.colorbar(scatter)
colorbar.set_label('Year', fontsize=11)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

3. Section 3: Multi-Panel Figures with Subplots

Create multiple plots in one figure for side-by-side comparisons:

```
# Create a 2x2 subplot grid
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Forest Area (top-left)
axes[0, 0].plot(df['Year'], df['Forest_Area_km2'], marker='o',
color='green', linewidth=2)
axes[0, 0].set_title('Forest Area Trend', fontweight='bold')
axes[0, 0].set_ylabel('Area (km²)')
axes[0, 0].grid(True, alpha=0.3)
```

```
# Plot 2: Deforestation Rate (top-right)
axes[0, 1].plot(df['Year'], df['Deforestation_Rate_%'], marker='s',
color='red', linewidth=2)
axes[0, 1].set_title('Deforestation Rate', fontweight='bold')
axes[0, 1].set_ylabel('Rate (%)')
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Protected Area (bottom-left)
axes[1, 0].bar(df['Year'], df['Protected_Area_km2'], color='blue',
alpha=0.7)
axes[1, 0].set_title('Protected Area', fontweight='bold')
axes[1, 0].set_ylabel('Area (km²)')
axes[1, 0].set_xlabel('Year')
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Plot 4: Population (bottom-right)
axes[1, 1].plot(df['Year'], df['Population']/1000, marker='^',
color='purple', linewidth=2)
axes[1, 1].set_title('Population', fontweight='bold')
axes[1, 1].set_ylabel('Population (thousands)')
axes[1, 1].set_xlabel('Year')
axes[1, 1].grid(True, alpha=0.3)

# Add overall title
fig.suptitle('Environmental and Demographic Metrics', fontsize=16,
fontweight='bold', y=0.995)

# Adjust spacing
plt.tight_layout()
plt.show()
```

4. Section 4: Customization and Styling

4.a. Colors, Labels, and Formatting

Make your plots publication-ready with careful styling:

```
# Professional-looking plot with custom styling
fig, ax = plt.subplots(figsize=(12, 7))

# Plot multiple lines
ax.plot(df['Year'], df['Forest_Area_km2']/1000,
        marker='o', linewidth=2.5, label='Forest Area', color='#2E7D32')
ax.plot(df['Year'], df['Protected_Area_km2']/1000,
        marker='s', linewidth=2.5, label='Protected Area',
color='#1976D2')

# Customize appearance
ax.set_xlabel('Year', fontsize=12, fontweight='bold')
ax.set_ylabel('Area (thousands km²)', fontsize=12, fontweight='bold')
ax.set_title('Forest Conservation Status', fontsize=14,
fontweight='bold', pad=20)
```

```
# Customize grid
ax.grid(True, alpha=0.4, linestyle='--', linewidth=0.7)
ax.set_axisbelow(True)

# Customize legend
ax.legend(loc='best', fontsize=11, frameon=True, shadow=True)

# Add subtle background color
ax.set_facecolor('#F5F5F5')
fig.patch.set_facecolor('white')

# Customize spines
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_linewidth(1.5)
ax.spines['bottom'].set_linewidth(1.5)

plt.tight_layout()
plt.show()
```

4.b. Colorblind-Friendly Palettes

Always consider accessibility when choosing colors:

```
# Colorblind-friendly color palette
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Data for comparison
categories = df['Year'].astype(str).values
values1 = df['Forest_Area_km2'].values
values2 = df['Deforestation_Rate_%'].values

# Plot 1: NOT colorblind-friendly (red-green)
axes[0].bar(categories[::2], values1[::2], color='red', label='Forest',
alpha=0.7)
axes[0].bar(categories[::2], values2[::2]*1000, color='green',
label='Deforestation', alpha=0.7)
axes[0].set_title('❌ Not Colorblind-Friendly\n(Red-Green)',
fontweight='bold')
axes[0].legend()
axes[0].grid(True, alpha=0.3, axis='y')

# Plot 2: Colorblind-friendly (blue-orange)
width = 0.35
x = np.arange(len(categories[::2]))
axes[1].bar(x - width/2, values1[::2], width, color='#0173B2',
label='Forest', alpha=0.8)
axes[1].bar(x + width/2, values2[::2]*1000, width, color='#DE8F05',
label='Deforestation', alpha=0.8)
axes[1].set_xticks(x)
axes[1].set_xticklabels(categories[::2])
axes[1].set_title('✓ Colorblind-Friendly\n(Blue-Orange)',
```

```
fontweight='bold')
axes[1].legend()
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# List of colorblind-friendly palettes
print("Recommended Colorblind-Friendly Palettes:")
print("- 'viridis', 'plasma', 'inferno' (perceptually uniform)")
print("- '#0173B2' (blue), '#DE8F05' (orange), '#029E73' (green),
'#CC78BC' (purple)")
print("- Use combinations like (blue, orange) or (blue, orange, green)")
```

5. Section 5: Saving and Exporting Plots

5.a. Save to Different Formats

```
# Create a simple plot for demonstration
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(df['Year'], df['Forest_Area_km2'], marker='o', linewidth=2,
color='green')
ax.set_title('Forest Area Over Time', fontsize=14, fontweight='bold')
ax.set_xlabel('Year', fontsize=12)
ax.set_ylabel('Area (km²)', fontsize=12)
ax.grid(True, alpha=0.3)

# Save in different formats
# PNG (good for screen, presentations)
fig.savefig('forest_trend.png', dpi=300, bbox_inches='tight',
facecolor='white')
print("✓ Saved: forest_trend.png (PNG - 300 DPI)")

# PDF (vector format, good for publishing)
fig.savefig('forest_trend.pdf', format='pdf', bbox_inches='tight',
facecolor='white')
print("✓ Saved: forest_trend.pdf (PDF - Vector)")

# SVG (editable vector format)
fig.savefig('forest_trend.svg', format='svg', bbox_inches='tight',
facecolor='white')
print("✓ Saved: forest_trend.svg (SVG - Editable)")

plt.show()

# Print save options reference
print("\nCommon save parameters:")
print("- dpi: 300 (print), 100-150 (screen/web)")
print("- bbox_inches='tight': Remove extra whitespace")
print("- facecolor: Set background color")
print("- format: 'png', 'pdf', 'svg', 'jpg', etc.")
```

6. Section 6: Practical Example - Creating a Publication-Quality Report

6.a. Scenario

Create a comprehensive analysis dashboard for environmental monitoring data that's ready to share with stakeholders.

```
# Create a comprehensive publication-ready dashboard
fig = plt.figure(figsize=(16, 10))
gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)

# Title
fig.suptitle('Environmental Monitoring Dashboard', fontsize=18,
fontweight='bold', y=0.98)

# Plot 1: Main trend
ax1 = fig.add_subplot(gs[0, :])
ax1.plot(df['Year'], df['Forest_Area_km2']/1000, marker='o',
linewidth=3,
         label='Forest Area', color='#2E7D32', markersize=8)
ax1.fill_between(df['Year'], df['Forest_Area_km2']/1000 - 100,
df['Forest_Area_km2']/1000 + 100,
                 alpha=0.2, color='#2E7D32')
ax1.set_ylabel('Forest Area (1000s km²)', fontweight='bold')
ax1.legend(fontsize=11)
ax1.grid(True, alpha=0.3)

# Plot 2: Bar chart
ax2 = fig.add_subplot(gs[1, 0])
subset_years = df[df['Year'].isin([2016, 2018, 2020, 2022])]
bars = ax2.bar(subset_years['Year'].astype(str),
subset_years['Deforestation_Rate_%'],
               color=['#D32F2F' if x > 2.7 else '#FFA726' for x in
subset_years['Deforestation_Rate_%']])
ax2.set_ylabel('Deforestation Rate (%)', fontweight='bold')
ax2.set_title('Annual Deforestation Rate', fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

# Plot 3: Scatter
ax3 = fig.add_subplot(gs[1, 1])
scatter = ax3.scatter(df['Protected_Area_km2']/1000,
df['Forest_Area_km2']/1000,
                      c=df['Year'], cmap='viridis', s=150, alpha=0.7,
edgecolors='black', linewidth=1.5)
ax3.set_xlabel('Protected Area (1000s km²)', fontweight='bold')
ax3.set_ylabel('Forest Area (1000s km²)', fontweight='bold')
ax3.set_title('Protected vs Total Forest', fontweight='bold')
cbar = plt.colorbar(scatter, ax=ax3)
cbar.set_label('Year', fontweight='bold')
ax3.grid(True, alpha=0.3)

# Plot 4: Area chart
```

```
ax4 = fig.add_subplot(gs[2, :])
ax4.fill_between(df['Year'], 0, df['Forest_Area_km2']/1000, alpha=0.4,
label='Forest', color='#2E7D32')
ax4.fill_between(df['Year'], df['Forest_Area_km2']/1000,
(df['Forest_Area_km2']+df['Protected_Area_km2'])/1000,
                 alpha=0.4, label='Protected Area', color='#1976D2')
ax4.set_xlabel('Year', fontweight='bold')
ax4.set_ylabel('Area (1000s km²)', fontweight='bold')
ax4.set_title('Stacked Area Chart', fontweight='bold')
ax4.legend(loc='upper left', fontsize=10)
ax4.grid(True, alpha=0.3)

plt.savefig('environmental_dashboard.png', dpi=300, bbox_inches='tight',
facecolor='white')
print("✓ Dashboard saved as 'environmental_dashboard.png'")
plt.show()
```

7. Section 7: Enhanced Visualization with Seaborn

Seaborn is built on top of matplotlib and provides high-level functions for creating beautiful statistical visualizations with minimal code.

7.a. Key Advantages of Seaborn

- Beautiful default themes and color palettes
- Statistical estimation and visualization
- Dataset-oriented API that works directly with DataFrames
- Specialized plots like heatmaps, violin plots, and pair plots
- Built-in support for categorical data

7.b. Seaborn Example 1: Heatmap for Correlation Matrix

Heatmaps are excellent for visualizing relationships between multiple variables:

```
# Calculate correlation matrix
corr = df.corr(numeric_only=True)

# Create heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix of Environmental Variables', fontsize=13,
fontweight='bold', pad=15)
plt.tight_layout()
plt.show()

print("Interpretation: Values close to +1 (dark red) indicate strong
positive correlation.")
print("Values close to -1 (dark blue) indicate strong negative
correlation.")
```

7.c. Seaborn Example 2: Regression Plot with Confidence Interval

Show relationships and trends with automatic statistical estimation:

```
# Regression plot with confidence interval
plt.figure(figsize=(10, 6))
sns.regplot(x='Protected_Area_km2', y='Forest_Area_km2', data=df,
            scatter_kws={'s': 100, 'alpha': 0.6},
            line_kws={'linewidth': 2, 'color': '#2E7D32'})

plt.title('Relationship: Protected Area vs Forest Area', fontsize=13,
fontweight='bold')
plt.xlabel('Protected Area (km²)', fontsize=11)
plt.ylabel('Forest Area (km²)', fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("The shaded area shows the 95% confidence interval for the
regression line.")
```

7.d. *Seaborn Example 3: Distribution and Relationship Pair Plot*

View all pairwise relationships in your dataset at once:

```
# Pair plot - shows all relationships
selected_cols = ['Year', 'Forest_Area_km2', 'Protected_Area_km2',
'Deforestation_Rate_%']
df_subset = df[selected_cols].copy()

# Scale for better visualization
df_subset['Forest_Area_km2'] = df_subset['Forest_Area_km2'] / 1000  #
Convert to thousands
df_subset['Protected_Area_km2'] = df_subset['Protected_Area_km2'] / 1000
df_subset = df_subset.rename(columns={
    'Forest_Area_km2': 'Forest (1000s km²)',
    'Protected_Area_km2': 'Protected (1000s km²)',
    'Deforestation_Rate_%': 'Deforestation (%)'
})

pair_plot = sns.pairplot(df_subset, diag_kind='hist', plot_kws={'alpha':
0.6, 's': 80},
                         diag_kws={'bins': 5, 'edgecolor': 'black'})

pair_plot.fig.suptitle('Pairwise Relationships: All Variables',
fontsize=14, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()

print("Each cell shows a different relationship.")
print("Diagonal shows distributions of individual variables.")
```

8. Section 8: Quick Reference and Best Practices

8.a. *When to Use Each Library*

Task	Best Choice	Why

Task	Best Choice	Why
Quick exploration	Pandas `.plot()`	Minimal code, immediate feedback
Fine-grained control	Matplotlib	Complete customization, precise positioning
Statistical visualizations	Seaborn	Built-in stats, beautiful defaults
Categorical data	Seaborn	Better handling of categories
Complex multi-panel layouts	Matplotlib	More control over GridSpec
Correlation/relationship analysis	Seaborn	Heatmaps, pair plots, regression

8.b. Pandas vs Matplotlib vs Seaborn

```
# PANDAS - Quick and simple
df.plot(x='Year', y='Value', kind='line')

# MATPLOTLIB - Fine control
fig, ax = plt.subplots()
ax.plot(df['Year'], df['Value'])
ax.set_title('My Title')
# ... extensive customization

# SEABORN - Beautiful statistics
sns.regplot(x='X', y='Y', data=df)
sns.heatmap(corr_matrix)
```

8.c. Pro Tips

1. **Always set a random seed** for reproducibility
2. **Use colorblind-friendly palettes** (viridis, husl, colorblind)
3. **Save at high DPI** (300 for print, 100-150 for web)
4. **Use vector formats** (PDF, SVG) for publications
5. **Keep it simple** - follow the 5 visualization pillars
6. **Label everything** - titles, axes, legends
7. **Remove chartjunk** - unnecessary decorations
8. **Use consistent styling** across related plots

---

9. Resources and Further Learning

9.a. Documentation

- **Matplotlib:** https://matplotlib.org/stable/index.html
- **Pandas Plotting:** https://pandas.pydata.org/docs/user_guide/visualization.html
- **Seaborn:** https://seaborn.pydata.org/

9.b. Recommended Palettes

- **Sequential (continuous data):** viridis, plasma, inferno, coolwarm
- **Diverging (data with center):** RdBu, RdYlBu, coolwarm
- **Qualitative (categories):** Set2, husl, deep, pastel

- **Colorblind-friendly:** viridis, cividis

9.c. Color Resources

- ColorBrewer: https://colorbrewer2.org/
- Colorblind simulator: https://www.color-blindness.com/coblis-color-blindness-simulator/

---

1. ✓ Setting up pandas, matplotlib, and seaborn
2. ✓ Creating basic plots (line, bar, scatter)
3. ✓ Building multi-panel layouts
4. ✓ Customizing colors, fonts, and styling
5. ✓ Saving publication-quality visualizations
6. ✓ Advanced statistical plots with Seaborn
7. ✓ Best practices for data visualization

Next Steps:

- Apply these techniques to your own geospatial data
- Combine with geopandas for spatial visualizations
- Explore interactive alternatives (Leafmap, Plotly)
- Create reusable plotting functions for your workflows

Happy visualizing!

```
### Seaborn Example 1: Heatmap for Correlation Matrix

Heatmaps are excellent for visualizing relationships between multiple
variables:
```